

PRACTICAL-05

AIM: Implementation of a knapsack problem using dynamic programming

ALGORITHM:

0/1 Knapsack Using Dynamic Programming

Algorithm:

I/p: no. of items (n), capacity (w), weight array ($w[i]$) & value ($v[i]$)

O/p: Maximum value & selected items.

step-1: Start

step-2: for $i = 1$ to n , $w[i]$ and $v[i]$ of each item

step-3: initialize, for $i = 0$ to n set: DP
 $dp[i][0] = 0$

step-4: for $j = 0$ to w , set:
 $dp[0][j] = 0$

step-5: for $i = 1$ to n do:
for $j = 1$ to w , do

step-6: If:
 $w[i] \leq j$

step-7: Then, $dp[i][j] = \max(v[i] + dp[i-1][j-w[i]], dp[i-1][j])$

else:
 $dp[i][j] = dp[i-1][j]$

step-8: max value, $dp[n][w]$

step-9: set $j = w$

step-10: for $i = n$ to 1 , do:
if $dp[i][j] = dp[i-1][j]$
Then, item i is Taken
 $j = j - w[i]$
Else, item i is NOT TAKEN

step-11: Return $dp[n][w]$

step-12: stop

PROGRAM:

```
#include<iostream>

using namespace std;

class Knapsack {

private:

    int w[10];      //weights
    int v[10];      // values
    int n;          // number of items
    int W;          // capacity
    int dp[10][100]; // dp table

public:

    // — Get Input —
    void getInput() {
        cout << "Enter number of items  : ";
        cin  >> n;

        cout << "Enter knapsack capacity : ";
        cin  >> W;

        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin  >> w[i] >> v[i];
        }
    }

    // — Build dp table —
    void solve() {
        // base case — row 0 and col 0
        for(int i = 0; i <= n; i++) dp[i][0] = 0;
        for(int j = 0; j <= W; j++) dp[0][j] = 0;
```

Name:B.Manoj Karthik

Enrolment number:92460118094

```
// fill table
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= W; j++) {
        if(w[i] <= j) {
            // take or not take
            dp[i][j] = max(
                v[i] + dp[i-1][j-w[i]],
                dp[i-1][j]
            );
        } else {
            // cannot take item i
            dp[i][j] = dp[i-1][j];
        }
    }
}

// — Print dp table —
void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "  ";
    for(int j = 0; j <= W; j++)
        cout << j << " ";
    cout << endl;
    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";
    }
}
```

```
    cout << endl;

}

}

// — Find selected items —

void findItems() {

    cout << "\nSelected Items:" << endl;

    int j = W;

    for(int i = n; i >= 1; i--) {

        if(dp[i][j] != dp[i-1][j]) {

            cout << "Item " << i

                << " (w=" << w[i]

                << ", v=" << v[i]

                << ") TAKEN " << endl;

            j = j - w[i];

        } else {

            cout << "Item " << i

                << " (w=" << w[i]

                << ", v=" << v[i]

                << ") NOT taken " << endl;

        }

    }

}

// — Show result —

void showResult() {

    cout << "\nMaximum Value = "

        << dp[n][W] << endl;

}
```

```
};  
  
// — MAIN —  
  
int main() {  
    Knapsack k;  
    k.getInput();  
    k.solve();  
    k.printTable();  
    k.showResult();  
    k.findItems();  
    return 0;  
}
```

OUTPUT:

```
Output  
  
Enter number of items: 4  
Enter knapsack capacity: 7  
Item 1 weight and value: 3 1  
Item 2 weight and value: 4 2  
Item 3 weight and value: 5 6  
Item 4 weight and value: 2 3  
  
DP Table:  
    0 1  2  3  4  5  6  7  
i=0 0 0  0  0  0  0  0  0  
i=1 0 0  0  1  1  1  1  1  
i=2 0 0  0  1  2  2  2  3  
i=3 0 0  0  1  2  6  6  6  
i=4 0 0  3  3  3  6  6  9  
  
Maximum Value = 9  
  
Selected Items:  
Item 4 (w=2, v=3) TAKEN  
Item 3 (w=5, v=6) TAKEN  
Item 2 (w=4, v=2) NOT taken  
Item 1 (w=3, v=1) NOT taken  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity

```

for i = 1 to n      → n times
{
  for j = 1 to w    → w times
  {
    if (w[i] ≤ j)
      dp[i][j] = max(v[i] + dp[i-1][j-w[i]], dp[i-1][j]);
    else
      dp[i][j] = dp[i-1][j];
  }
}

```

$T(n, w) = n \times w \times 1 + 1 + 1$
 $T(n, w) = n \times w$
 $T(n, w) = O(nw)$

Best case

In best case, condition is

$$w[i] \geq j$$

So, execute only else

But loop runs still:

outer loop - n times

inner loop - n times

$$\text{So, } T(n, w) = n \times w$$

$$\boxed{\text{best case} = O(nw)}$$

∴ In best case, all DP table cells must be checked

Worst case

In worst case, condition

$w[i] \leq j$ is correct in many times

then perform:

$$v[i] + dp[i-1][j-w[i]] \text{ } dp[i-1][j] \rightarrow 1$$

$$\max(\dots)$$

Average case

In Average case,

$$w[i] \leq j$$

Sometimes: $w[i] > j$

So both if & else occurs

But with take constant time

outer loop → n times

inner loop → w times

$$\text{So, } T(n, w) = n \times w \times 1$$

$$\boxed{\text{Average case} = O(nw)}$$

for (i = 1 to n) - n times

for (j = 1 to w) - w times

$$\text{So, } T(n, w) = n \times w \times 1 \times 1$$

$$\boxed{\text{Worst case} = O(nw)}$$

∴ Every item is checked for every possible capacity.

CONCLUSION:

The 0/1 Knapsack problem was successfully implemented using Dynamic Programming. The algorithm finds the maximum value within the given knapsack capacity and identifies the selected items. Its best, average and worst-case time complexity is $O(nw)$.

Name: B. Manoj Karthik

Enrolment number: 92460118094